Machine Learning Practical Python Code

https://gemini.google.com/app/11aab163572e1f8f

User prompt: Give me the codes from exp 1 to 9 from this pdf don't include outputs

Response: **EXP1**

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

data = {
    'Student': ['a', 'b', 'c', 'd', 'e'],
    'hours': [1, 2, 3, 4, 5],
    'marks': [45, 50, 55, 60, 70]
}
df = pd.DataFrame(data)

mean = np.mean(df['marks'])
median = np.median(df['marks'])

plt.figure()
plt.plot(df['hours'], df['marks'], marker='o')
plt.title('Hours vs Marks')
plt.xlabel('Hours')
plt.ylabel('Marks')
plt.grid(True)
plt.show()

plt.figure()
sns.boxplot(x=df['marks'])
plt.title('Marks Distribution')
plt.show()
```

EXP2

```
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.metrics import mean_squared_error, accuracy_score

# Simple Linear Regression
X1 = [[1], [2], [3], [4]]
y1 = [2, 4, 5, 8]
model1 = LinearRegression().fit(X1, y1)
pred1 = model1.predict(X1)

plt.scatter(X1, y1, color='black')
plt.plot(X1, pred1, color='red')
plt.title("Simple Linear Regression")
plt.show()

# Multiple Linear Regression
X2 = [[1, 2], [2, 4], [3, 6], [4, 8]]
y2 = [3, 6, 9, 12]
model2 = LinearRegression().fit(X2, y2)
pred2 = model2.predict(X2)

plt.scatter(y2, pred2, color='black')
plt.plot(y2, y2, color='red')
plt.title("Multiple Linear (Actual vs Predicted)")
plt.show()

# Logistic Regression
X3 = [[1], [2], [8], [9]]
y3 = [0, 0, 1, 1]
model3 = LogisticRegression().fit(X3, y3)
pred3 = model3.predict(X3)

plt.scatter(X3, y3, color='black')
plt.plot(X3, pred3, color='red')
plt.title("Logistic Regression")
plt.show()
```

EXP3

```
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier, plot_tree

X = [[0, 0], [1, 1], [2, 2], [3, 3]]
y = [0, 0, 1, 1]

model = DecisionTreeClassifier()
model.fit(X, y)

test_data = [[1.5, 1.5]]
model.predict(test_data)

plt.figure(figsize=(6, 4))
plot_tree(
    model,
    feature_names=["Feature 1", "Feature 2"],
    class_names=["Low", "High"],
    filled=True
)
plt.show()
```

EXP4

```
from sklearn.svm import SVC, SVR, OneClassSVM

# SVM for Classification
X_clf = [[1, 2], [2, 3], [3, 4], [6, 7], [7, 8], [8, 9]]
y_clf = [0, 0, 0, 1, 1, 1]
model_clf = SVC()
model_clf.fit(X_clf, y_clf)
model_clf.predict([[5, 6]])

# SVM for Regression
X_reg = [[1], [2], [3], [4], [5]]
y_reg = [2, 4, 6, 8, 10]
model_reg = SVR()
model_reg.fit(X_reg, y_reg)
model_reg.predict([[6]])

# SVM for Outlier Detection
X_train = [[1], [1.1], [1.2], [1.1]]
model_outlier = OneClassSVM(gamma='auto')
model_outlier.fit(X_train)
test_data = [[1.15], [9.9]]
model_outlier.predict(test_data)
```

EXP5

```
from sklearn.ensemble import AdaBoostClassifier
from xgboost import XGBClassifier

# AdaBoost
X_ada = [[1, 2], [2, 3], [3, 4], [6, 7], [7, 8], [8, 9]]
y_ada = [0, 0, 0, 1, 1, 1]
model_ada = AdaBoostClassifier(n_estimators=50)
model_ada.fit(X_ada, y_ada)
model_ada.predict([[5, 6]])
model_ada.score(X_ada, y_ada)

# XGBoost
X_xgb = [[0], [1], [2], [3]]
y_xgb = [0, 0, 1, 1]
model_xgb = XGBClassifier(n_estimators=10, random_state=0)
model_xgb.fit(X_xgb, y_xgb)
model_xgb.predict([[1.5]])
```

EXP6

```
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

X = [[1, 2], [2, 3], [3, 2], [8, 8], [9, 8], [8, 9]]
model = KMeans(n_clusters=2, random_state=0, n_init=10)
model.fit(X)

plt.scatter([i[0] for i in X], [i[1] for i in X], c=model.labels_)
plt.scatter(model.cluster_centers_[:, 0],
```

```
            model.cluster_centers_[:, 1],
            marker='X', s=200)
plt.show()
```

EXP7

```
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA

X = [[25, 45_000, 12],
     [40, 120_000, 3],
     [22, 38_000, 25],
     [55, 180_000, 8],
     [31, 72_000, 15],
     [48, 140_000, 4]]

pca = PCA(n_components=2)
X_reduced = pca.fit_transform(X)

plt.scatter(X_reduced[:, 0], X_reduced[:, 1])
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("PCA Scatter Plot")
plt.show()
```

EXP8

```
import numpy as np

q = np.zeros((3, 2))
reward = [[-1, 0], [-1, 10], [0, 10]]
alpha = 0.1
gamma = 0.9
epsilon = 0.3

for i in range(1000):
    state = np.random.randint(0, 3)
    action = np.argmax(q[state])
    next_state = min(state + 1, 2)
    q[state, action] += 0.1 * (reward[state][action] + 0.9 * np.max(q[next_state]) - q[state, action])
```

EXP9

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV, RandomizedSearchCV

X, y = load_iris(return_X_y=True)
model = DecisionTreeClassifier()

params = {
    'max_depth': [2, 3, 4, 5],
    'criterion': ['gini', 'entropy']
}

grid = GridSearchCV(model, params, cv=5)
grid.fit(X, y)

random = RandomizedSearchCV(model, params, cv=5, n_iter=4, random_state=0)
random.fit(X, y)
```
